Глава 13. **Библиотека стандартных контейнерных классов**

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

Стандартные контейнерные классы STD

- **последовательные** - обеспечивают хранение конечного количества *однотипных* величин в виде непрерывной последовательности;

- **ассоциативные** контейнеры обеспечивают быстрый доступ к данным по ключу, построены на основе сбалансированных деревьев.

13.1 Последовательные контейнеры

- **`vector`** – динамический массив — структура, эффективно реализующая произвольный доступ к элементам, добавление в конец и удаление из конца;
- **`list`** – линейный список – эффективно реализует вставку и удаление элементов в/из произвольного места и не эффективно - произвольный доступ к элементам;
- **`stack`** – стек – эффективно реализует добавление в конец и удаление из конца;
- **`queue`** – очередь – эффективно реализует добавление в конец и удаление из начала;
- **`deque`** – двусторонняя очередь (дек) – эффективно реализует произвольный доступ к элементам, добавление в оба конца и удаление из обоих концов;
- **`priority_queue`** – очередь, сортированная по приоритетам.

Операции последовательных контейнеров

Операция	Метод	vector	list	stack	queue	deque
Вставка в начало	push_front()	-	+	+	-	+
Удаление из начала	pop_front()	-	+	+	+	+
Вставка в конец	push_back()	+	+	-	+	+
Удаление из конца	pop_back()	+	+	-	-	+
Вставка в произвольное место	insert()	+*	+	-	-	-
Удаление из произвольного места	erase()	+*	+	-	-	-
Произвольный доступ	[ ], at (с проверкой)	+	+*	+	+	+

Вектор vector

Синтаксис:

```
template <class Type, class Allocator = allocator<Type>>
   class vector
```

где Allocator – объект, отвечающий за распределение памяти

Примеры вызова конструкторов:

```
а) vector<one> m;          // пустой вектор
б) vector<one> m1(10);  /* вектор из 10 объектов класса one
                           (работает конструктор one без параметров) */
в) vector<int> v1(10, 1);// вектор из 10 единичных элементов
г) vector<int> v2(v1);  // вектор, равный вектору v1
```

Дополнительные методы:

- резервирование места: void reserve (size n)
- создание объекта сразу в векторе (без временного объекта):

  void emplace_back(Параметры_объекта)

Основные типы, используемые в шаблонах

`value_type` - тип элемента контейнера;

`size_type` - тип индексов элементов и т. д.;

`iterator` - тип "итератор" – указатель на элемент;

`const_iterator` - тип "константный итератор" – используется для неизменных данных;

`reverse_iterator` - обратный итератор – для просмотра от конца к началу;

`const_reverse_iterator` - константный обратный итератор;

`reference` - ссылка на элемент;

`const_reference` - константная ссылка на элемент;

Итераторы

Работа с элементами контейнеров ведется через итераторы и ссылки.

Итератор – объект, обладающий свойствами указателя, для доступа к элементу его необходимо разыменовать.

Пример объявление итератора:

```
vector<double> ::iterator i;
```

Доступ к элементу через итератор:

```
*i
```

Перемещение итератора:

```
++i, i++
```

Операции над итераторами

Категория итератора	Допустимые операции	Контей-неры
Входной (input) – перемещается вперед и допускает только чтение элементов	`x = *i,` `++i, i++`	все
Выходной (output) – перемещается вперед и допускает запись элементов	`*i = x,` `++i, i++`	все
Прямой (forward) – допускает запись и чтение элементов при движении вперед.	`x = *i, *i = x,` `++i, i++`	все
Реверсивный (reverse) – допускает запись и чтение элементов при движении назад.	`x =*i, *i = x,` `--i, i--`	все
Двунаправленный (bidirectional) – работает вперед и назад	`X = *i,` `++i,i++,--i,i--`	все

Произвольного доступа
(random access)
	`x = *i, *i = x,`
	`--i, i--`
	`i + n, i – n,`
	`i += n, i -= n`
	`i < j, i > j,`
	`i <= j, i >= j`
		все,
		кроме list

Методы получения адресов для инициализации итераторов

Указатель на первый:

```
iterator begin()
const_iterator begin() const
```

Указатель на следующий за последним:

```
iterator end()
const_iterator end() const
```

Указатель на первый при обратном просмотре:

```
reverse_iterator rbegin()
const_reverse_iterator rbegin() const
```

Указатель на элемент, следующий за последним, при обратном просмотре:

```
reverse_iterator rend()
const_reverse_iterator rend() const
```

Пример использования шаблона Динамический массив (Ex13.1)

```
#include <fstream>
#include <iostream>
#include <vector>
using namespace std;
int main() {
   ifstream in ("inp.txt");
   vector<int> v;   // инстанцирование шаблона Динамический
                    // массив
  int x;
  while ( in >> x )
      v.push_back(x); // добавление элемента в конец
   for (vector<int>::iterator i = v.begin();
                                   i != v.end(); ++i)
      cout << *i << " ";
  return 0;
}
```

Пример 13_2 Массив объектов, включающих русские буквы

```
#include <fstream>
#include <iostream>
#include <vector>
#include <string>
using namespace std;
class A {
    int num;
    string st;
  public:
    A(int n,string s):num(n){
        st = s;
    }
    int get_num(){ return num; }
    string get_s(){ return st;  }
};
```

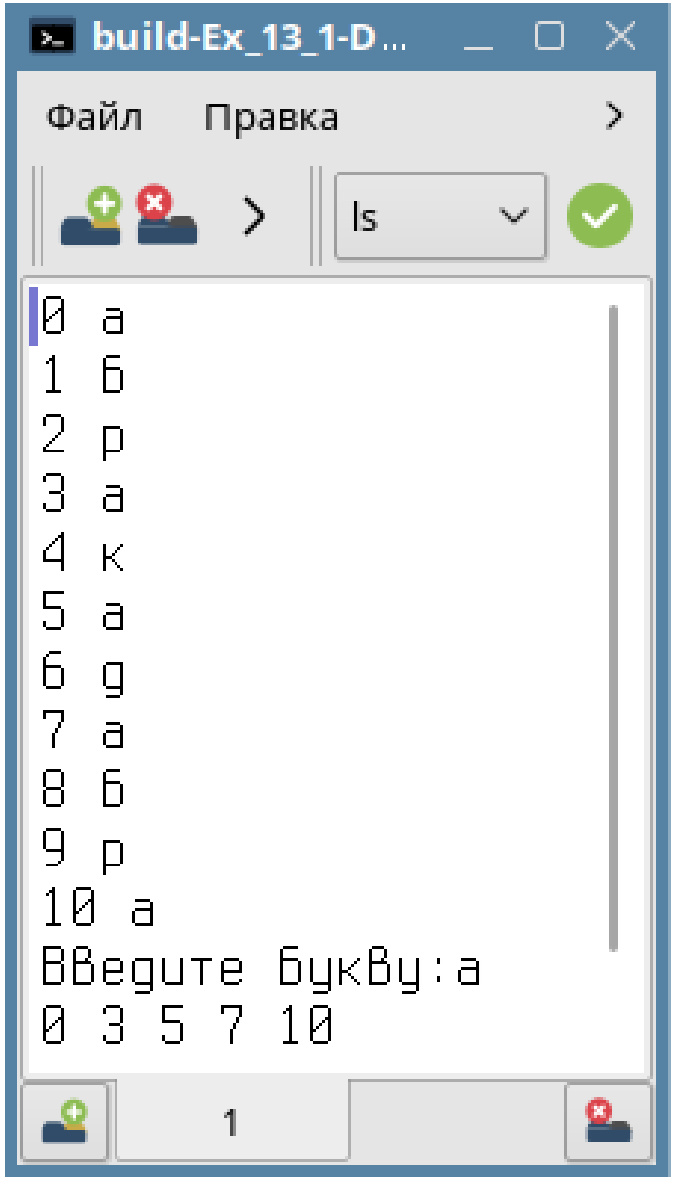
Пример. Продолжение

```
int main() {
    string s;
    int k = 0;
    bool fl = false;
    ifstream in ("/home/user/projects/Ex_13_1/inp.txt");
    vector<A> v;        // инстанцирование шаблона
    v.reserve(25);      // резервирование места
    while (in >> s)     // русских букв, записанных через пробел
        v.emplace_back(k++,s);  // создание элемента в векторе
    // вывод объектов на экран
    for (vector<A>::iterator i = v.begin();
                                    i != v.end(); ++i)
    cout << (*i).get_num() << " " << (*i).get_s() <<
                                                  endl;
```

Пример. Окончание

```
cout << "Введите букву:" ;
    cin >> s ;
    for (A a:v)     // цикл «для каждого»
        if (a.get_s() == s) {
            cout << a.get_num() << ' ';
            fl = true;
        }
    if (!fl) cout << "Такой буквы нет.";
    cout << endl;
    return 0;
}
```

Пример. Результат



13.2 Ассоциативные контейнеры

- **map** – словарь с уникальными ключами,
- **multimap** – словарь ключей с дубликатами,
- **set** - множество,
- **multiset** – мультимножество,
- **bitset** – битовое множество (набор битов).

Словари часто называют также *ассоциативными массивами* или *отображениями*.

Словарь построен на основе пар значений, первое из которых представляет собой ключ для идентификации элемента, а второе — собственно элемент.

Компонентная функция **`find()`** ищет элемент контейнера по заданному ключу.

Типы данных:

`key_type` - тип ключа (для ассоциативных контейнеров);

`key_compare` - тип результата сравнения ключей.

Пример работы с ассоциативным контейнером

Ex13_03

```
#include <fstream>
#include <iostream>
#include <string>
#include <map>
using namespace std;

int main()
{
   map <string, long> m;     // объявление словаря
   string str;
   long num;
```

Пример работы с ассоциативным контейнером

```
ifstream in("/home/user/Ex13_02/text.txt");
while (in >> num)          // ввод числа из текстового файла
{
    in.get();              // пропуск пробела или маркера
    getline(in, str);      // ввод строки
    m[str] = num;
    cout << str << "  " << num << endl;
}
```

Файл text.txt

```
122 Petr
156 Nicolay
354 Vadim
```

Результат:

```
Petr  122
Nicolay  156
Vadim  354
```

Работа со словарем

```
m:
Martin  213
Nicolay  156
Petr  122
Vadim  354
Second: Nicolay  156
```

```
// дополнение словаря
m["Martin"] = 213;

// вывод словаря
cout << "m:" << endl;
map <string, long> :: iterator i;
for (i = m.begin(); i != m.end(); i++)
    cout << (*i).first << "  " << (*i).second << endl;

// вывод второй пары
i = m.begin();
i++;
cout << "Second: ";
cout << (*i).first << "  " << (*i).second << endl;
```

Поиск элемента по ключу

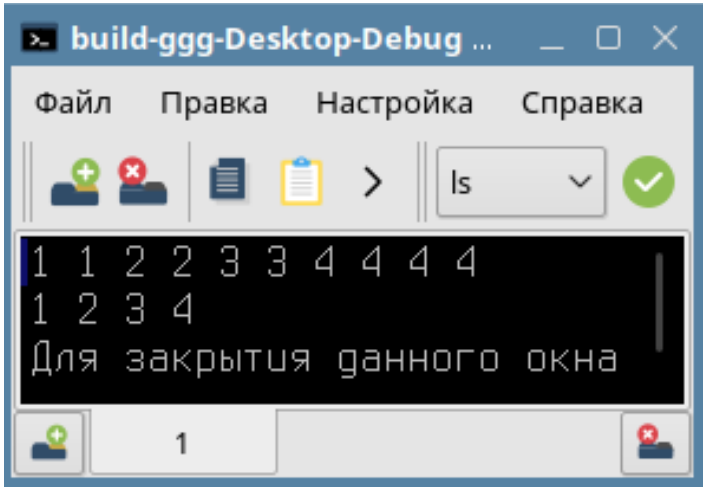
```
// вывод элемента по ключу

  cout << "Name: " << endl;
  getline(cin, str);                // ввод ключа
  if (m.find(str) != m.end())       // поиск значения по ключу
      cout << m[str]<<endl;
  else
      cout << "Error" ;
  return 0;
}
```

```
Name:
Vadim
354
Для продолжения нажмите любую клавишу . . .
```

Множество

Ex13_04




```
#include <iostream>
#include <vector>
#include <set>
using namespace std;
int main()
{
    vector<int> v(10); // вектор
    for (int i = 0; i < 10; i++) cin >> v[i];
    set<int> s;          // множество
    for (int i = 0; i < 10; i++) s.insert(v[i]);
    for (set<int>::iterator p = s.begin();
                                        p != s.end(); ++p)
        cout << *p << ' ';
    cout << endl;
    return 0;
}
```